

TECHNICAL WHITE PAPER

The Essential Operations of Computing

A mathematical bridge across binary logic, statistics, and machine learning

bits → arithmetic → vectors → inference → learning

August 2026

Abstract

Computing systems may look very different at the application level, yet they are built from a compact operational vocabulary. A processor compares, masks, shifts, and adds bits; numerical software accumulates products and reductions; statistics estimates quantities from samples; machine learning evaluates a model, measures error, differentiates that error, and updates parameters. This paper identifies the most consequential operations spanning those layers. It emphasizes mathematical notation, binary representations, numerical examples, and the relationships that let the same hardware primitives support both a Boolean gate and a modern learning system.

Notation and the Unifying View

An operation is a state transformation. In the broadest useful form, a program or model maps input state x to output state y , possibly using parameters θ . The input may be a word of bits, a vector of measurements, a matrix of features, or a distribution. The essential question is not whether an operation belongs to math, statistics, or machine learning; it is what it transforms, what it costs, and how it composes with other operations.

$$y = f(x; \theta), \quad b \in \{0, 1\}, \quad x \in \mathbb{R}^n, \quad A \in \mathbb{R}^{m \times n}$$

Notation. b denotes a bit; x is a vector; A is a matrix; θ denotes learnable parameters; $\odot$ is elementwise multiplication; $\|x\|_2$ is the Euclidean norm; and $\mathbb{1}[\text{condition}]$ is an indicator that equals one when its condition is true.

Table 1. The operation stack: a small set of primitives, reused at progressively higher levels.

Layer	Canonical state	Dominant operations	Result
Digital logic	$b \in \{0,1\}^w$	NOT, AND, OR, XOR, add, shift	flags and integers
Numerical computing	$x \in \mathbb{R}^n$	multiply-add, sum, max, matrix multiply	vectors and transforms
Statistics	$x_1, \dots, x_n$	count, mean, variance, likelihood, sample	estimates and uncertainty
Machine learning	x, θ	linear map, nonlinearity, loss, gradient, update	predictions and trained parameters

A useful mental model is layered composition. Boolean operations implement integer arithmetic. Arithmetic implements vector and matrix operations. Those operations implement likelihoods, losses, transforms, and optimizers. The result is a continuous ladder from physical bits to learned behavior.

Binary Foundations

Representation, arithmetic, and Boolean control

A bit is a two-state symbol $b_i \in \{0,1\}$. An unsigned word of width w represents a nonnegative integer by positional weighting. The rightmost bit has weight 2^0 ; each move left doubles the weight. This simple representation is the substrate of addressing, flags, loop counters, integer arithmetic, and packed low-precision data.

$$u = \sum_{i=0}^{w-1} b_i 2^i \quad ; \quad 13_{10} = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 1101_2$$

Binary addition. Addition proceeds from least-significant to most-significant bit while carrying overflow to the next position. In a fixed-width register, any carry beyond the left edge is discarded for unsigned modular arithmetic.

$$\begin{array}{r} 0101_2 \quad (5) \\ + 0011_2 \quad (3) \\ \hline 1000_2 \quad (8) \end{array}$$

Signed integers normally use two's complement. In a w -bit word, negation is produced by inverting every bit and adding one. The signed range is -2^{w-1} through $2^{w-1} - 1$. This choice lets the same adder perform both addition and subtraction.

$$\begin{aligned} +5 &= 0101_2 \\ -5 &= \text{invert}(0101_2) + 1 = 1010_2 + 1 = 1011_2 \end{aligned}$$

$$1001_2 + 1011_2 = 1\ 0100_2 \rightarrow 0100_2 \quad (9 - 5 = 4, \text{ using four bits})$$

Table 2. Boolean primitives. XOR is especially important for parity, change detection, cryptography, and binary arithmetic.

A	B	A AND B	A OR B	A XOR B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Bitwise operations act on whole words in parallel. AND selects positions permitted by a mask, OR sets requested flags, XOR reports unequal bits, and shifts multiply or divide unsigned values by powers of two when overflow and rounding are controlled.

$$\begin{aligned} 1100_2 \text{ AND } 1010_2 &= 1000_2 & 1100_2 \text{ XOR } 1010_2 &= 0110_2 \\ 0011\ 0110_2 \ll 1 &= 0110\ 1100_2 & 0011\ 0110_2 \gg 2 &= 0000\ 1101_2 \end{aligned}$$

Finite precision is part of the operation

Hardware does not compute over ideal real numbers. An integer word has a fixed range; for an unsigned w -bit addition, overflow is arithmetic modulo 2^w . Floating-point arithmetic uses a sign, exponent, and fraction, then rounds the result to a finite set of representable values. A concise error model is shown below, where δ is a small rounding error bounded by the format and the operation.

$$\text{fl}(a \circ b) = (a \circ b)(1 + \delta), \quad \circ \in \{+, -, \times, \div\}$$

Practical implication. The decimal value 0.1 has a repeating binary expansion, so it is usually approximated in binary floating point. Equality tests, large sums, and extreme exponentials therefore need numerically stable formulations rather than assumptions of exact real arithmetic.

Linear Algebra: Weighted Sums at Scale

Most high-throughput scientific and ML computation is a structured form of multiply-add. The scalar fused multiply-add operation accumulates a product into a running total: $\text{acc} \leftarrow \text{acc} + ab$. A dot product reduces many such terms to one scalar; matrix multiplication computes many related dot products at once. GPUs, vector units, and dedicated tensor processors are optimized around this pattern.

$$s = w^T x = \sum_{i=1}^n w_i x_i \quad ; \quad C = AB, \quad C_{ij} = \sum_k A_{ik} B_{kj}$$

Vector example. For a binary feature vector $x = (1, 0, 1, 1)$ and weights $w = (0.50, -0.25, 0.75, 0.50)$, the score is $w^T x = 0.50 + 0 + 0.75 + 0.50 = 1.75$. The same operator underlies a linear regression prediction, one neuron in a neural network, a similarity score, and a finite-difference stencil.

$$\begin{aligned} x &= [1, 0, 1, 1] & w &= [0.50, -0.25, 0.75, 0.50] \\ \text{score} &= \sum w_i x_i = 1.75 \end{aligned}$$

Matrix example. A matrix multiply is not merely a larger multiplication; it is a tiled family of dot products with strong data reuse.

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 1 & 1 \end{bmatrix}$$

The transpose A^T exchanges rows and columns; norms summarize magnitude; and normalization makes scale comparable. The Euclidean norm is used in distance, regularization, and cosine similarity.

$$\|x\|_2 = \sqrt{\sum_i x_i^2}, \quad \cos(x, y) = (x^T y) / (\|x\|_2 \|y\|_2)$$

A bridge from bits to dot products

When data and weights are restricted to the binary values -1 and $+1$, a dot product can be evaluated using bitwise logic. Encode $+1$ as bit 1 and -1 as bit 0. Equal signs contribute $+1$ and unequal signs contribute -1 , so the count of equal positions obtained by XNOR replaces many multiplications.

$$\mathbf{x}^T \mathbf{w} = 2 \cdot \text{popcount}(\text{XNOR}(\mathbf{b}_x, \mathbf{b}_w)) - n$$

$$\begin{aligned} \mathbf{b}_x &= 1011_2 & \mathbf{b}_w &= 1101_2 \\ \text{XNOR}(\mathbf{b}_x, \mathbf{b}_w) &= 1001_2 & \text{popcount} &= 2 \\ 2 \cdot 2 - 4 &= 0 & & \text{(the corresponding } \pm 1 \text{ dot product)} \end{aligned}$$

This identity illustrates why binary neural networks and packed similarity search can be much more efficient than naïve scalar arithmetic. The trade-off is representational precision: binary values are cheap to process but cannot express every useful weight or activation.

Statistical Operations: From Samples to Evidence

Counts, moments, and reductions

Statistics begins with aggregation. Count, sum, minimum, maximum, and mean are reductions: they turn many observations into a smaller summary. For observations $x_1, \dots, x_n$, the sample mean estimates central tendency and the sample variance estimates dispersion. The denominator $n - 1$ gives the usual unbiased sample-variance estimator under independent sampling.

$$\bar{x} = (1/n) \sum_i x_i \quad ; \quad s^2 = (1/(n - 1)) \sum_i (x_i - \bar{x})^2$$

Binary sample example. For the four Bernoulli observations (0, 1, 1, 0), the count of ones is $\text{popcount}(0110_2) = 2$. Therefore $\bar{x} = 2/4 = 0.5$; the population variance is 0.25 and the sample variance is $1/3$. For binary data, the mean is also the observed event rate.

$$\text{Cov}(X, Y) = (1/(n - 1)) \sum_i (x_i - \bar{x})(y_i - \bar{y})$$

Covariance measures joint deviation and motivates the covariance matrix used in principal-component analysis, Gaussian models, portfolio risk, and uncertainty propagation. Correlation rescales covariance by standard deviations, making it unitless but not causal.

Likelihood, Bayes' rule, and information

Likelihood connects a proposed model to observed data. For a Bernoulli process with k ones in n trials, the probability model is $p^k(1 - p)^{n-k}$. Computation almost always uses the log-likelihood: sums of logs are more stable and more convenient to differentiate than products of small probabilities.

$$L(p) = p^k(1 - p)^{n-k} \quad ; \quad \ell(p) = k \log p + (n - k) \log(1 - p)$$

Bayesian example. Suppose a rare condition has prior probability $P(D) = 0.01$. A binary test has sensitivity $P(+|D) = 0.90$ and false-positive rate $P(+|\text{not } D) = 0.05$. After a positive result, the posterior probability is only about 15.4%, because false positives are numerous when the base rate is low.

$$P(D|+) = (0.90 \cdot 0.01) / [(0.90 \cdot 0.01) + (0.05 \cdot 0.99)] = 0.1538$$

Information-theoretic operations quantify uncertainty and discrepancy. Entropy is the average information in a random variable. For a fair binary variable, $H = 1$ bit; for a highly predictable variable it is lower. Cross-entropy and Kullback-Leibler divergence later reappear as central ML loss functions.

$$H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p) \quad ; \quad H(0.5) = 1 \text{ bit}, H(0.9) \approx 0.469 \text{ bits}$$

Calculus and Optimization: How Models Improve

A model is useful only if its parameters can be adjusted to improve an objective. Calculus supplies local sensitivity: the derivative describes how one output changes with one input; the gradient stacks derivatives of a scalar objective with respect to many parameters; and the Jacobian generalizes derivatives to vector-valued functions. These are the operational language of fitting and control.

$$\nabla \theta L(\theta) = [\partial L / \partial \theta_1, \dots, \partial L / \partial \theta_d]^T \quad ; \quad J_i = \partial f_i / \partial x_j$$

The chain rule makes deep networks trainable. If $z = f(x)$ and $L = g(z)$, then the sensitivity of L to x is the product of local sensitivities. Backpropagation is an efficient organization of this repeated chain-rule computation, reusing intermediate values produced during the forward pass.

$$dL/dx = (dL/dz)(dz/dx)$$

Gradient-descent example. For a one-parameter linear model $\hat{y} = wx$ and squared loss $L = 1/2(\hat{y} - y)^2$, take $x = 2$, $y = 6$, and $w = 1$. The prediction is 2, the gradient is $(2 - 6) \cdot 2 = -8$, and a step of size $\eta = 0.1$ moves the weight to 1.8. The next prediction is 3.6, closer to the target.

$$w \leftarrow w - \eta \partial L / \partial w = 1 - 0.1(-8) = 1.8$$

Optimization is also a numerical-computing problem. Very large logits may overflow $\exp(z)$, while very small probabilities may underflow to zero. Stable implementations subtract the maximum before exponentiating. This changes neither the softmax nor the log-sum-exp result, because the same constant is applied to every term.

$$\log \sum_i \exp(z_i) = m + \log \sum_i \exp(z_i - m), \quad m = \max_i z_i$$

Core Machine-Learning Operations

Linear scores, nonlinearities, and loss

The basic supervised-learning block is compact: compute a linear score, transform it into an output, compare the output with a target, and optimize the parameters. Feature scaling is often performed first so that measured units do not dominate merely because they have larger numerical ranges.

$$x'_j = (x_j - \mu_j) / \sqrt{(\sigma_j^2 + \epsilon)} \quad ; \quad z = w^T x + \beta$$

Logistic example. Let $x = (1, 0, 1)$, $w = (0.7, -0.4, 0.6)$, and $\beta = -0.5$. The score is $z = 0.8$. The sigmoid converts that score to a probability-like output of approximately 0.690.

$$\sigma(z) = 1/(1 + \exp(-z)), \quad \sigma(0.8) \approx 0.690$$

For a multiclass prediction, softmax maps logits to nonnegative values that sum to one. With $z = (2, 1, 0)$, the probabilities are approximately (0.665, 0.245, 0.090). If class one is correct, cross-entropy is $-\log(0.665) \approx 0.408$. The logarithm heavily penalizes confident wrong predictions.

$$\text{softmax}(z)_i = \exp(z_i) / \sum_j \exp(z_j) \quad ; \quad \text{CE}(y, p) = -\sum_i y_i \log p_i$$

Activation functions introduce nonlinearity. ReLU is a max operation that keeps positive signals and suppresses negative ones. Masks are binary gates that can select features, prevent attention to invalid positions, or implement regularization such as dropout.

$$\text{ReLU}(z) = \max(0, z), \quad \text{ReLU}([-2, 0, 3]) = [0, 0, 3] \quad ; \quad x' = m \odot x, m_i \in \{0, 1\}$$

Convolution, transforms, and attention

Convolutional layers are local weighted sums. In many ML libraries the operation called convolution is technically cross-correlation, with the kernel applied in a fixed orientation. It detects local patterns such as edges, short temporal events, or molecular motifs. The discrete Fourier transform instead projects a sequence into sinusoidal frequency components; the fast Fourier transform computes the same result more efficiently.

$$y_i = \sum_j k_j x_{i+j} \quad ; \quad x = [1, 0, 1, 1, 0], k = [1, -1] \rightarrow y = [1, -1, 0, 1]$$

$$X_N = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N} \quad ; \quad \text{DFT}([1, 0, 1, 0]) = [2, 0, 2, 0]$$

Attention is a three-stage operation: score the relation between queries and keys, normalize those scores with softmax, then form a weighted sum of values. It has become central in sequence, image, multimodal, and scientific models because the weighted sum is conditioned on context.

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})V$$

Operational interpretation. Attention is still composed from familiar parts: matrix multiplication for similarity, max/exp/sum for normalization, and another matrix multiplication for aggregation. Its distinctive behavior comes from the data-dependent weights, not from a wholly new arithmetic primitive.

Dataflow Operations and Computational Cost

Mathematical operations are inseparable from data movement. A computation must read inputs, place intermediate values in registers or memory, and write results. On modern accelerators, moving large

arrays can cost more time and energy than the arithmetic itself. Algorithms are therefore designed around locality, batching, tiling, and parallel reductions as much as around formulas.

$$\text{map: } y_i = f(x_i) \quad ; \quad \text{reduce: } s = \bigoplus_i x_i \quad ; \quad \text{scan: } s_i = \bigoplus_{j \leq i} x_j$$

Map applies the same operation independently over items. Reduce combines items with an associative operator such as sum, max, AND, or XOR. Scan produces running partial results, useful for cumulative sums, stream processing, and parallel algorithms. Sort, gather, scatter, and random sampling are similarly basic dataflow operations: they rearrange or select data before numerical kernels consume it.

Table 3. Typical asymptotic costs. Constant factors, memory traffic, sparsity, and hardware layout often determine real runtime.

Operation	Representative expression	Typical cost
Reduction	$\sum_i x_i$ or $\max_i x_i$	$O(n)$ work; parallel tree depth $O(\log n)$
Sorting	$\text{argsort}(x)$	$O(n \log n)$ comparison work
Matrix-vector	$y = Ax$	$O(mn)$
Dense matrix multiply	$C = AB$	$O(mnk)$
FFT	$\text{DFT}(x)$	$O(n \log n)$
Dense attention	$\text{softmax}(QK^T)V$	$O(T^2d)$ for sequence length T

The practical hierarchy is therefore: choose the right mathematical operation, organize it into a numerically stable algorithm, then lay out the data so hardware can execute it efficiently. A theoretically elegant expression can be slow if it creates unnecessary random reads or materializes an enormous intermediate matrix.

An End-to-End Binary-to-Learning Example

Consider a four-bit observation $b = 1011_2$. Treat the bits as features $x = (1, 0, 1, 1)$ and pass them through a logistic classifier with $w = (0.50, -0.25, 0.75, 0.50)$ and $\beta = -0.40$. The pipeline starts with binary storage but ends with a probability, a loss, and a parameter update.

$$z = w^T x + \beta = 0.50 + 0 + 0.75 + 0.50 - 0.40 = 1.35 \quad ; \quad p = \sigma(z) \approx 0.794$$

Suppose the true label is $y = 1$. Binary cross-entropy is $L = -\log p \approx 0.231$. For this model, the gradient of the loss with respect to the weights is $(p - y)x$. Because $p - y \approx -0.206$, the active binary features receive negative gradients, increasing their positive weights under a standard descent update.

$$\begin{aligned} \partial L / \partial w &= (p - y)x \approx [-0.206, 0, -0.206, -0.206] \\ \eta = 0.1: \quad w' &= w - \eta \partial L / \partial w \approx [0.5206, -0.25, 0.7706, 0.5206] \end{aligned}$$

This one example exposes the entire stack: bit representation, memory reads, multiply-add reduction, exponential and division in sigmoid, logarithm in loss, subtraction and multiplication in the gradient, and memory writes for the new parameters. Larger ML systems add more dimensions and more layers, but they do not escape this operational foundation.

Conclusion: The Minimum Useful Operational Vocabulary

The most important operations are best understood as a connected set rather than a flat list. At the hardware boundary are Boolean logic, integer addition, shifts, comparisons, and memory access. Numerical computing builds on multiply-add, reduction, matrix multiplication, transforms, and stable nonlinear functions. Statistics adds estimation, likelihood, sampling, conditioning, and information measures. Machine learning adds parameterized composition, loss evaluation, gradients, and optimization updates.

A strong practical foundation comes from learning to recognize the same few operations in different disguises: an XOR is a difference detector; a popcount is a binary reduction; a dot product is a weighted evidence accumulator; a softmax is normalized exponentiation; a likelihood is a product made tractable by logs; and backpropagation is repeated application of the chain rule. Once these identities are clear, both low-level computing and high-level ML architectures become much easier to analyze, implement, optimize, and verify.

Selected References

Bishop, C. M. Pattern Recognition and Machine Learning. Springer, 2006.

Cover, T. M., and Thomas, J. A. Elements of Information Theory, 2nd ed. Wiley, 2006.

Golub, G. H., and Van Loan, C. F. Matrix Computations, 4th ed. Johns Hopkins University Press, 2013.

Kingma, D. P., and Ba, J. Adam: A Method for Stochastic Optimization. arXiv:1412.6980, 2015.

Vaswani, A., et al. Attention Is All You Need. Advances in Neural Information Processing Systems, 2017.